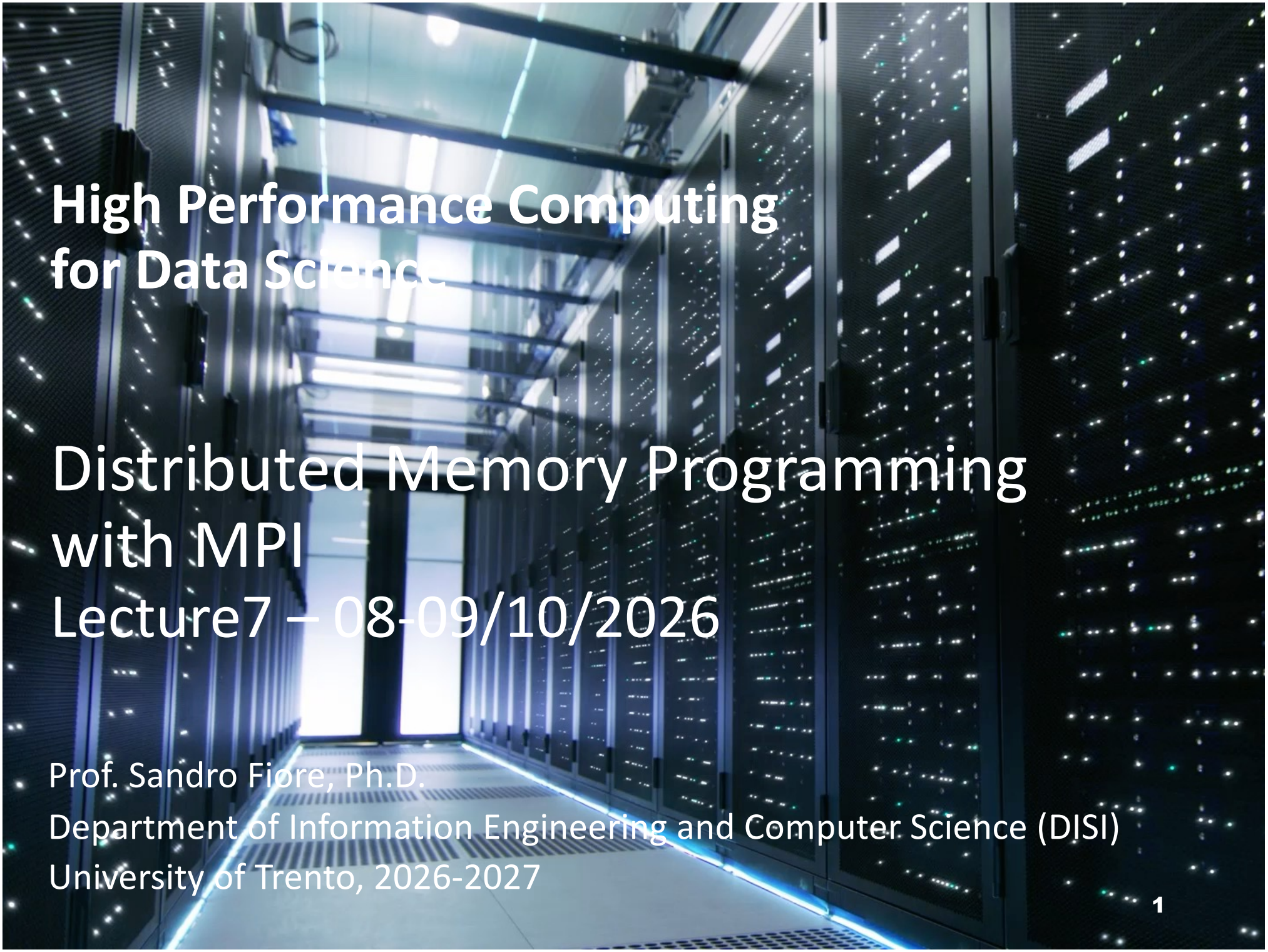


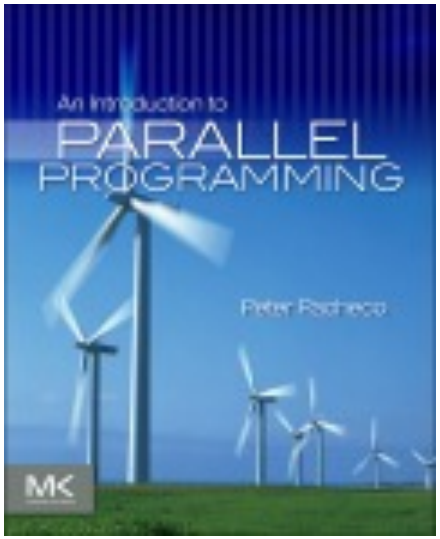
# High Performance Computing for Data Science

# Distributed Memory Programming with MPI
# Lecture7 – 08-09/10/2026

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

# Today's outline and material

- Recap
- Exercise: ring (open)
- Job submission (sequential today)
- Trapezoidal rule




## Today's material

Book
*An Introduction to Parallel Programming by Peter Pacheco*

Chapter 3
*Distributed Memory Programming with MPI*

# A few info before we start

- Account request: please proceed if you haven't done it so far
- Project teams (still this week to register)
- October 16 visit to the HPC Center in Povo2

- Speakers

  *Confirmed:*

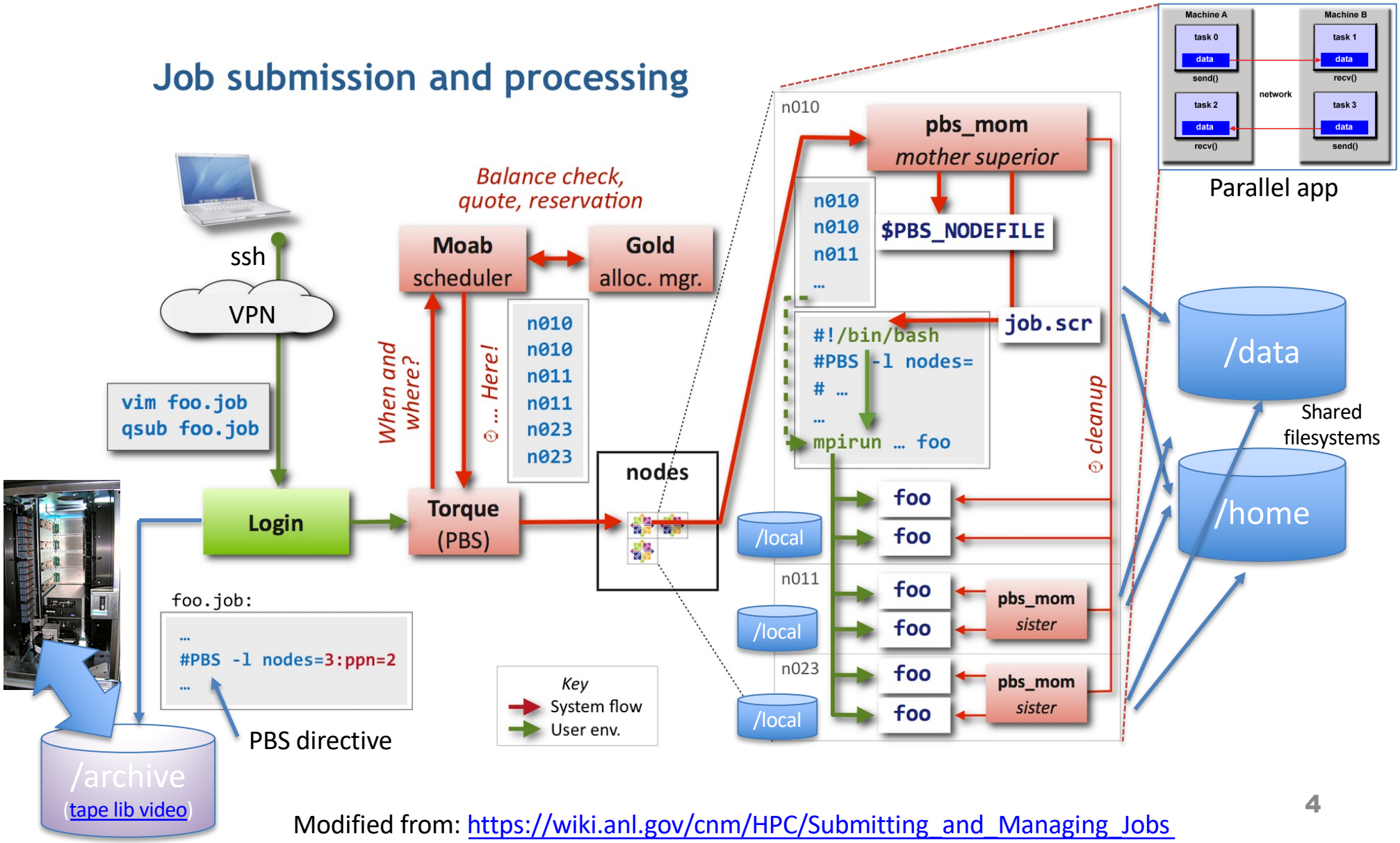
  – *Wes Brewer (Oak Ridge National Lab)*
  – *Valentine Anantharaj (Oak Ridge National Lab)*
  – *Davide Salomoni (ICSC)*

  *Pending:*

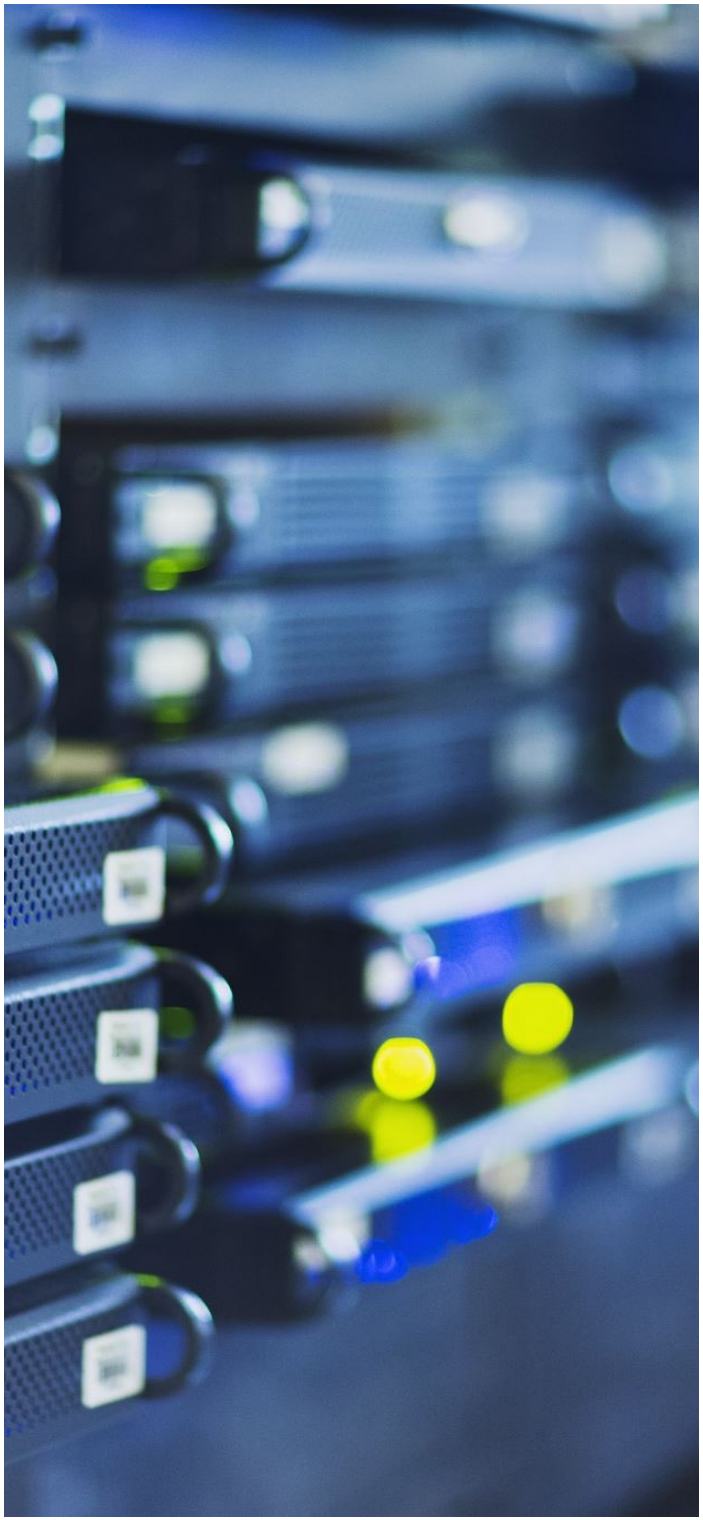
  – *Carole Goble (University of Manchester)*
  – *Kyle Chard (University of Chicago)*

# Submitting a job on a cluster

## End-to-end job submission/mng workflow




Modified from: https://wiki.anl.gov/cnm/HPC/Submitting_and_Managing_Jobs



# Virtual Private Network @ UniTrento

- Access to the HPC cluster @ UniTrento is provided via Virtual Private Network (VPN)

Info about VPN@UniTrento:

VPN

- https://servicedesk.unitn.it/goauth/en?s=S00180

MFA idp

- https://servicedesk.unitn.it/goauth/en?kb=KB0011764

Login Node

- https://servicedesk.unitn.it/goauth/en?kb=KB0013882

Cluster Policy

- https://servicedesk.unitn.it/goauth/en?kb=KB0010398

# Unitn HPC cluster usage conditions

English

**3. Home directory quota check**

3.1 Every week a check of the used space is perfor
exceeded.

3.2 In the latter case, the user must promptly reduc
reasoned request to benefit, for a limited period of
may be granted, denied or withdrawn based on the

3.3 After 4 unsuccesful warnings, non-compliant us
home directory will still be possible via FTPs servic
or remove their data.

**4. Home directory of users with expired accounts**

4.1 Users must remove all data from their home dir
cluster expires. Access to the home directory by us
via FTPs service (here the instructions).

4.2 On a weekly basis, a check is carried out that id
whose access to the cluster has expired, and an em
data that the user want to keep via FTPs and to ren

4.3 To safeguard the resources available on the syst
home will be removed **without any possibility of re**

**5. Emergency**

In the event of exceptional situations, when the infrastructure risks to stop operations due to the exhaustion of available space on the volume that contains the users' homes, after discussion with the Rector's Delegate for Services and IT technologies, the following extraordinary measures can be adopted:

5.1 It will be possible to intervene, even without the approval of the interested parties, on the home directories of users who have exceeded the quota granted, also through their removal.

5.2 It will be possible to intervene on the home directories of users whose access to the cluster has expired, also by deleting them before the period indicated in point 4.3.

**6. Prohibitions**

6.1 The use of HPC resources for activities other than teaching and research is strictly prohibited.

6.2 Calculations, simulations, etc. can only be started via the **qsub** command. In particular:

- It is forbidden to execute any job on the head-nodes of the cluster (hpc-head-n1 and hpc-head-n2);
- it is forbidden to access the compute nodes via ssh.

6.3 The jobs submitted by each single user are visible by all users of the cluster; in addition, some information relating to the jobs (name, date and time of submission and execution, resources requested, etc.), are collected and used by the cluster accounting system for statistical purposes and for reporting. For these reasons, in order to protect privacy, job names must in no way contain information that can be traced back to people. In cases where this may occur, pseudo anonymization techniques of the job names must be used (e.g. genetic_test_john_brown.sh can become genetic_test_user_4451.sh).

6.4 Failure to comply with the prohibitions 6.1, 6.2 and 6.3, barring cases whose gravity is such as to warrant other measures (disciplinary of the University or reporting to the authorities), provide for:

- at the first occurrence, an informal warning by email;
- in case of reiteration after the first warning, the case will be presented to the Rector's Delegate for IT Services and Technologies, who will arrange the measures to be taken, up to the prohibition of accessing the cluster.

https://servicedesk.unitn.it/goauth/en?kb=KB0010398

# Accessing the HPC Cluster

**Accessing the HPC Cluster**

To access the University HPC cluster (HPC3), connect via SSH to the following address:

- hpc3-login.unitn.it

**Linux and macOS**

From a terminal, run the following command:

- ssh university_username@hpc3-login.unitn.it

**Windows**

Use an SSH client such as **PuTTY** or **MobaXTerm**. Enter your University username as the **username**, your University Services password as the **password**, and hpc3-login.unitn.it as the **host address**.

**VPN**

If you are connecting from outside the University network, you must activate the VPN service before connecting to the cluster.

# Visit to the Data Center (Oct 16)



# PBS User Guide



ALTAIR

ONLY FORWARD

Altair® PBS Professional® 2025.2.0

Pdf on Moodle

# Hello world MPI program

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  // Initialize the MPI environment. The two arguments to MPI Init are not
  // currently used by MPI implementations, but are there in case future
  // implementations might need the arguments.
  MPI_Init(NULL, NULL);

  // Get the number of processes
  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size);

  // Get the rank of the process
  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

  // Print off a hello world message
  printf("Hello world from process rank %d out of %d processors\n", world_rank, world_size);

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

# Communicators

- A collection of processes that can send messages to each other.
- MPI_Init defines a communicator that consists of all the processes created when the program is started.
- Called MPI_COMM_WORLD.

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Message matching

```
MPI_Send(send_buf_p, send_buf_sz, send_type, dest, send_tag,
         send_comm);
```

*r*

*MPI_Send*

*src = q*



*MPI_Recv*

*dest = r*

```
MPI_Recv(recv_buf_p, recv_buf_sz, recv_type, src, recv_tag,
         recv_comm, &status);
```

*q*

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Let's move back to our MPI program

```
1  #include <stdio.h>
2  #include <string.h>  /* For strlen             */
3  #include <mpi.h>     /* For MPI functions, etc */
4
5  const int MAX_STRING = 100;
6
7  int main(void) {
8     char       greeting[MAX_STRING];
9     int        comm_sz;  /* Number of processes */
10    int        my_rank;  /* My process rank     */
11
12    MPI_Init(NULL, NULL);
13    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
14    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
15
16    if (my_rank != 0) {
17       sprintf(greeting, "Greetings from process %d of %d!",
18             my_rank, comm_sz);
19       MPI_Send(greeting, strlen(greeting)+1, MPI_CHAR, 0, 0,
20             MPI_COMM_WORLD);
21    } else {
22       printf("Greetings from process %d of %d!\n", my_rank, comm_sz);
23       for (int q = 1; q < comm_sz; q++) {
24          MPI_Recv(greeting, MAX_STRING, MPI_CHAR, q,
25             0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
26          printf("%s\n", greeting);
27       }
28    }
29
30    MPI_Finalize();
31    return 0;
32 }  /* main */
```




Copyright © 2010, Elsevier Inc. All rights Reserved

# Final considerations

- MPI requires that messages be ***nonovertaking***
- Potential Pitfalls in the send/receive
  - For each send we must have a *corresponding* receive
  - Tags too!
  - Destination and source processes to be properly set in the send/receive
  - Otherwise, we might have hanging processes!


Copyright © 2010, Elsevier Inc. All rights Reserved

# Input

- Most MPI implementations only allow process 0 in MPI_COMM_WORLD access to stdin.
- Process 0 must read the data (scanf) and send to the other processes.

```
. . .
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

Get_data(my_rank, comm_sz, &a, &b, &n);

h = (b-a)/n;
. . .
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Copying a file from your local machine to the cluster

You can use the scp command:

Syntax:

```
scp yourfile your_account@login_node:your_home_directory
```

Example:

```
scp readme.txt sandro.fiore@hpc3-login.unitn.it:/home/sandro.fiore
```

A video will be made available on moodle about scp + tar commands

# Useful tools: git

Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

You might find convenient to host your hands-on and project code on a github repo

https://git-scm.com/

https://git-scm.com/book/en/v2

https://github.com/

# Login to the HPC cluster

- Turn on VPN
- Check with ping if the login node is reachable
- `sfiore$ ping hpc3-login.unitn.it`

  …

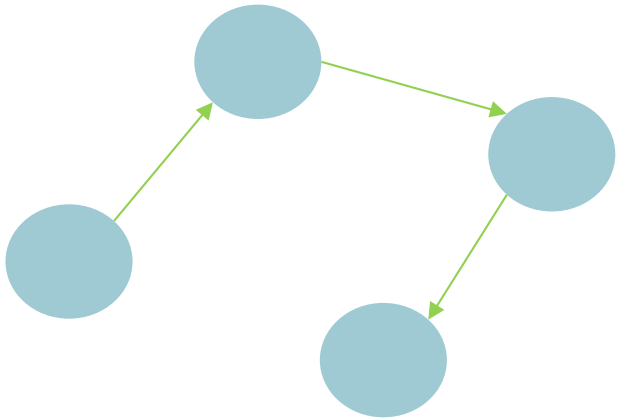
- Then run ssh using your ‘username’ e pwd (your UniTrento credentials):

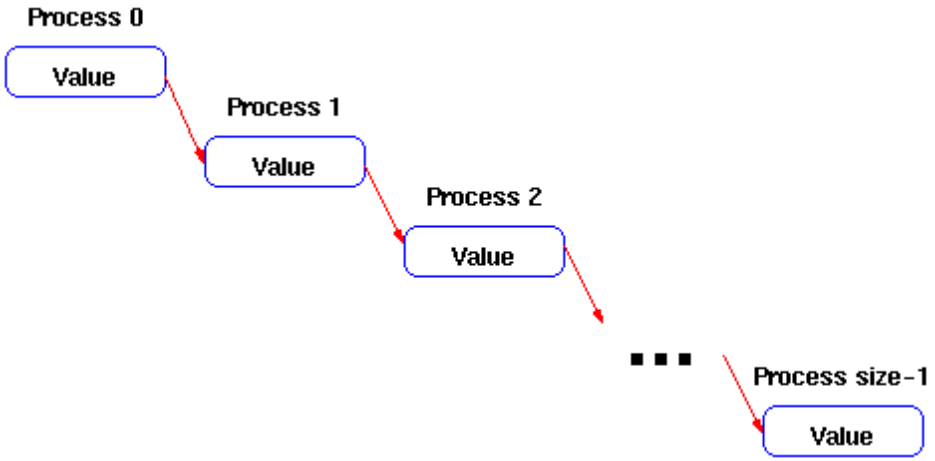
  `ssh sandro.fiore@hpc3-login.unitn.it`

# Communication hands-on

## Implement a ring by using send and receives

Write a program that takes data from process zero and sends it to all of the other processes by sending it in a ring. That is, process i should receive the data and send it to process i+1, until the last process is reached.





# Communication hands-on (solution)

```
#include <stdio.h>
#include "mpi.h"

int main( argc, argv )
int argc;
char **argv;
{
    int rank, value, size;
    MPI_Status status;

    MPI_Init( &argc, &argv );

    MPI_Comm_rank( MPI_COMM_WORLD, &rank );
    MPI_Comm_size( MPI_COMM_WORLD, &size );
    do {
        if (rank == 0) {
            scanf( "%d", &value );
            MPI_Send( &value, 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD );
        }
        else {
            MPI_Recv( &value, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD,
                      &status );
            if (rank < size - 1)
                MPI_Send( &value, 1, MPI_INT, rank + 1, 0, MPI_COMM_WORLD );
        }
        printf( "Process %d got %d\n", rank, value );
    } while (value >= 0);

    MPI_Finalize( );
    return 0;
}
```

# Module environment

```
Module [list | avail | load | unload]
```

- *list*: it lists the modules already loaded
- *load* <module> : it loads a module
- *unload* <module>: it unloads a module
- *avail*: it lists the modules available on the cluster

# Basics of PBS CLI (I)

## Sequential and parallel job submission

- To submit a job: **qsub** <*script*>
  - *You get back a job_ID*
- To monitor a job: **qstat** <*job_ID*>
  - *You get back job status info*
- To cancel a job: **qdel** <*job_ID*>
  - *Action job cancelled*

# Basics of PBS CLI (II)
## Job submission (sequential job)

**[sandro.fiore@hpc3-login0 ~]$ more test1.sh**

```
#!/bin/bash

#PBS -l select=1:ncpus=1:mpiprocs=1:mem=2gb

# set max execution time
#PBS -l walltime=0:00:30

# set the queue
#PBS -q shortCPUQ

echo "hostname: " && hostname
```

**[sandro.fiore@hpc3-login0 ~]$ qsub test1.sh**

**[sandro.fiore@hpc3-login0 ~]$ qstat <ID>**

# Homework - Communication

Design a broadcast solution by using send and receive calls

What we expect: a process “p” wants to share the content of a variable “a” with all the other processes.

# Question ?

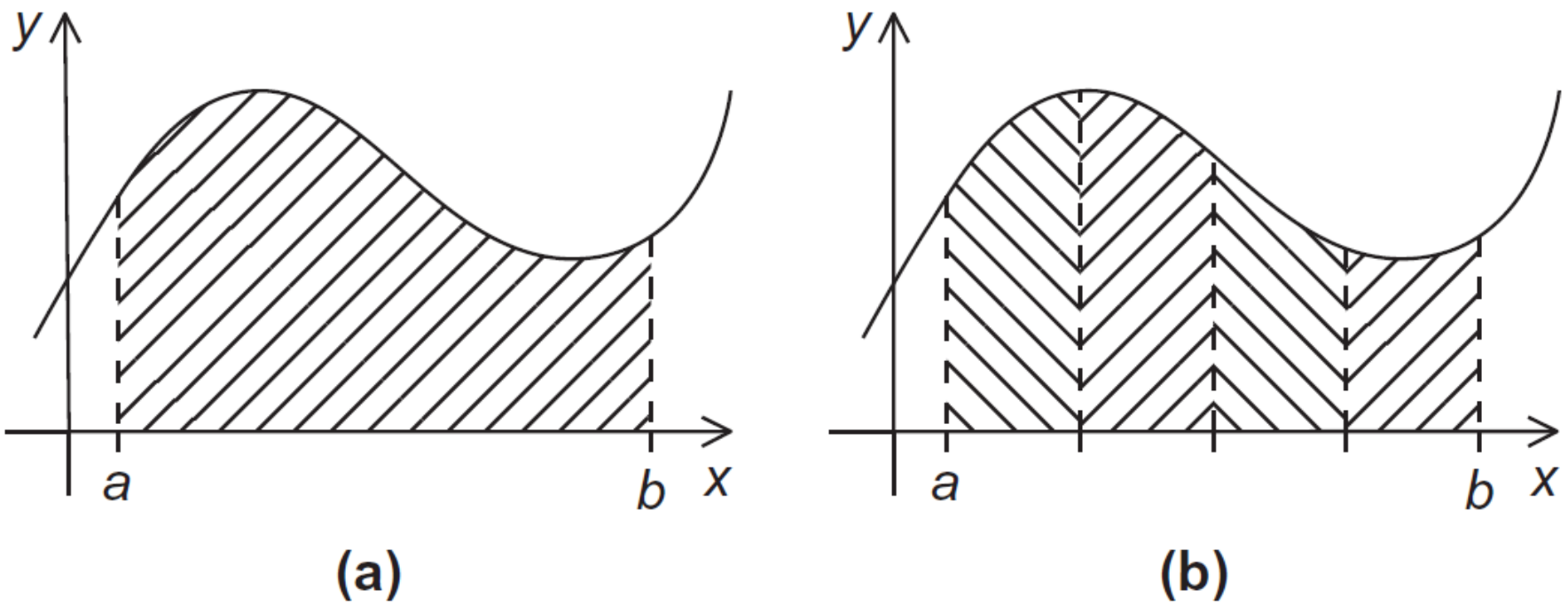
# High Performance Computing for Data Science

MPI Communication
Lecture 7 - 08/10/2026

## The Trapezoidal rule in MPI

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

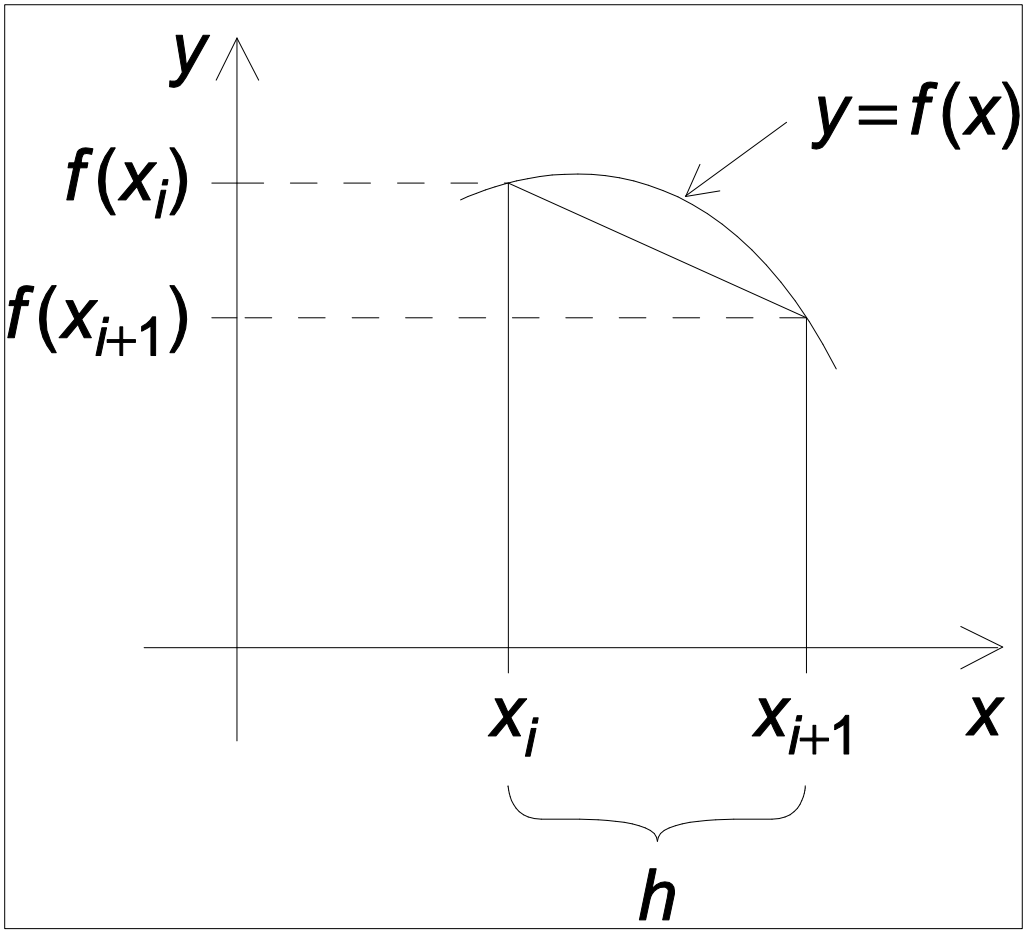
# The Trapezoidal Rule




*We can use the trapezoidal rule to approximate the area between the graph of a function, y=f(x), two vertical lines, and the x-axis*
*The basic idea is to divide the interval on the x-axis into n equal subintervals.*


Copyright © 2010, Elsevier Inc. All rights Reserved

# One trapezoid




*Then we approximate the area lying between the graph and each subinterval by a trapezoid whose base is the subinterval, whose vertical sides are the vertical lines through the endpoints of the subinterval, and whose fourth side is the secant line joining the points where the vertical lines cross the graph.*

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# The Trapezoidal Rule

$$\text{Area of one trapezoid } = \frac{h}{2}[f(x_i) + f(x_{i+1})]$$

$$h = \frac{b-a}{n}$$

$$x_0 = a,\ x_1 = a+h,\ x_2 = a+2h,\ \ldots,\ x_{n-1} = a+(n-1)h,\ x_n = b$$

$$\text{Sum of trapezoid areas } = h[f(x_0)/2 + f(x_1) + f(x_2) + \cdots + f(x_{n-1}) + f(x_n)/2]$$

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Pseudo-code for a serial program

$$\text{Sum of trapezoid areas} = h[f(x_0)/2 + f(x_1) + f(x_2) + \cdots + f(x_{n-1}) + f(x_n)/2]$$

```
/* Input:  a, b, n */
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++) {
    x_i = a + i*h;
    approx += f(x_i);
}
approx = h*approx;
```

MORGAN KAUFMANN

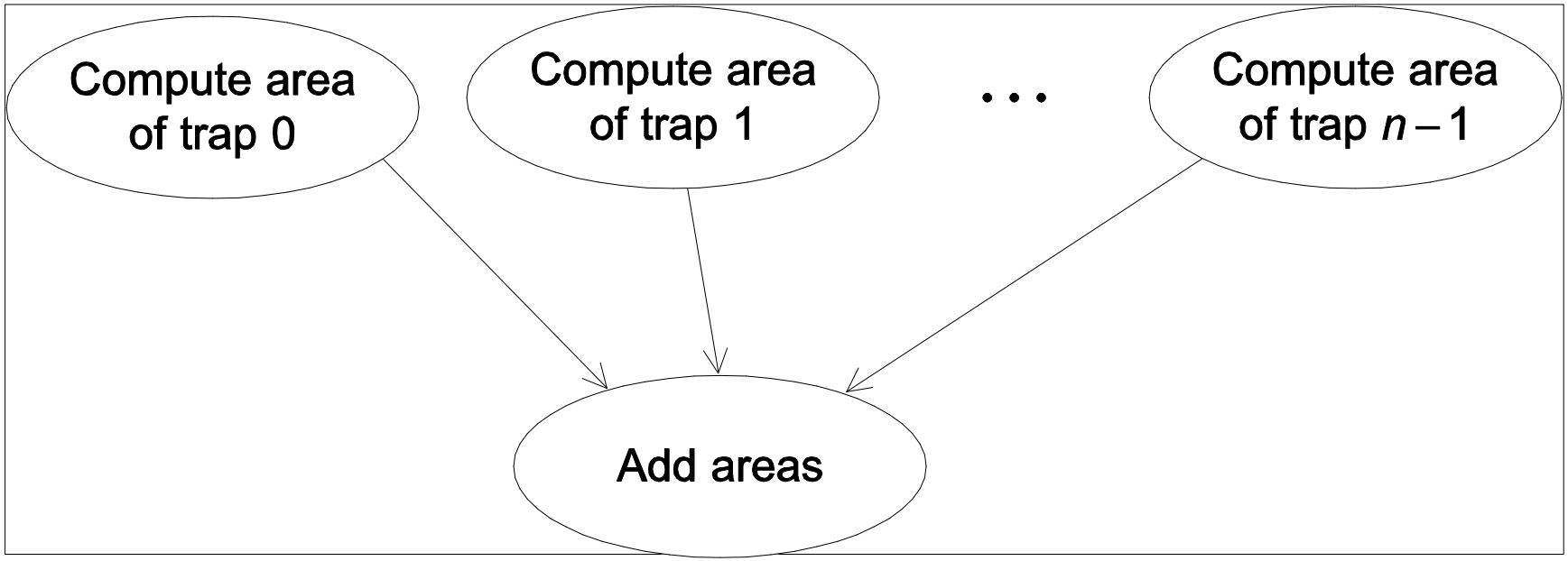
Copyright © 2010, Elsevier Inc. All rights Reserved

# Parallelizing the Trapezoidal Rule

1. Partition problem solution into tasks.
2. Identify communication channels between tasks.
3. Aggregate tasks into composite tasks.
4. Map composite tasks to cores.

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Tasks and communications for Trapezoidal Rule




MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Parallel pseudo-code

```
1     Get a, b, n;
2     h = (b-a)/n;
3     local_n = n/comm_sz;
4     local_a = a + my_rank*local_n*h;
5     local_b = local_a + local_n*h;
6     local_integral = Trap(local_a, local_b, local_n, h);
7     if (my_rank != 0)
8        Send local_integral to process 0;
9     else /* my_rank == 0 */
10       total_integral = local_integral;
11       for (proc = 1; proc < comm_sz; proc++) {
12          Receive local_integral from proc;
13          total_integral += local_integral;
14       }
15    }
16    if (my_rank == 0)
17       print result;
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# First version (1)

```
1  int main(void) {
2     int my_rank, comm_sz, n = 1024, local_n;
3     double a = 0.0, b = 3.0, h, local_a, local_b;
4     double local_int, total_int;
5     int source;
6
7     MPI_Init(NULL, NULL);
8     MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
9     MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
10
11    h = (b-a)/n;           /* h is the same for all processes */
12    local_n = n/comm_sz;   /* So is the number of trapezoids  */
13
14    local_a = a + my_rank*local_n*h;
15    local_b = local_a + local_n*h;
16    local_int = Trap(local_a, local_b, local_n, h);
17
18    if (my_rank != 0) {
19       MPI_Send(&local_int, 1, MPI_DOUBLE, 0, 0,
20             MPI_COMM_WORLD);
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# First version (2)

```
21      } else {
22          total_int = local_int;
23          for (source = 1; source < comm_sz; source++) {
24              MPI_Recv(&local_int, 1, MPI_DOUBLE, source, 0,
25                  MPI_COMM_WORLD, MPI_STATUS_IGNORE);
26              total_int += local_int;
27          }
28      }
29
30      if (my_rank == 0) {
31          printf("With n = %d trapezoids, our estimate\n", n);
32          printf("of the integral from %f to %f = %.15e\n",
33              a, b, total_int);
34      }
35      MPI_Finalize();
36      return 0;
37  } /*  main  */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# First version (3)

```
/* Input:  a, b, n */
h = (b-a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++) {
   x_i = a + i*h;
   approx += f(x_i);
}
approx = h*approx;
```

```
1  double Trap(
2        double left_endpt  /* in */,
3        double right_endpt /* in */,
4        int    trap_count  /* in */,
5        double base_len    /* in */) {
6     double estimate, x;
7     int i;
8
9     estimate = (f(left_endpt) + f(right_endpt))/2.0;
10    for (i = 1; i <= trap_count-1; i++) {
11       x = left_endpt + i*base_len;
12       estimate += f(x);
13    }
14    estimate = estimate*base_len;
15
16    return estimate;
17 } /*  Trap  */
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Input

- Most MPI implementations only allow process 0 in MPI_COMM_WORLD access to stdin.
- Process 0 must read the data (scanf) and send to the other processes.

```
. . .
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);

Get_data(my_rank, comm_sz, &a, &b, &n);

h = (b-a)/n;
. . .
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Function for reading user input

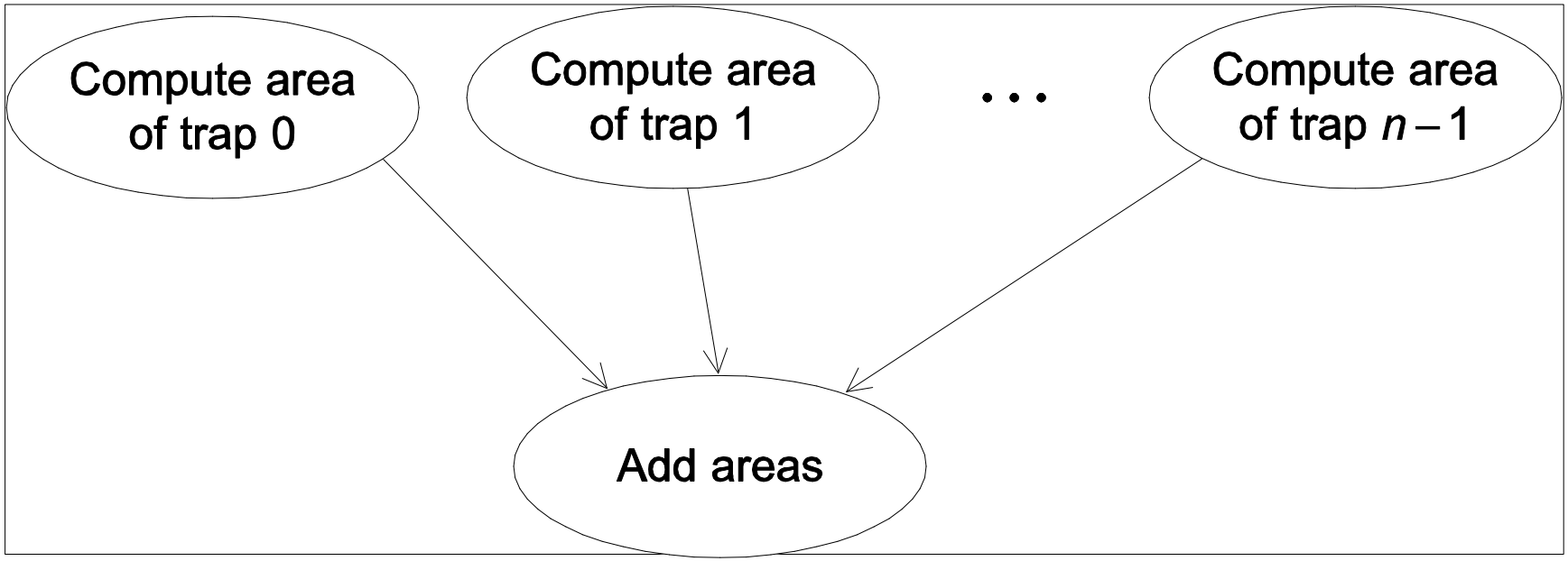
```
void Get_input(
      int       my_rank   /* in  */,
      int       comm_sz   /* in  */,
      double*   a_p       /* out */,
      double*   b_p       /* out */,
      int*      n_p       /* out */) {
   int dest;

   if (my_rank == 0) {
      printf("Enter a, b, and n\n");
      scanf("%lf %lf %d", a_p, b_p, n_p);
      for (dest = 1; dest < comm_sz; dest++) {
         MPI_Send(a_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
         MPI_Send(b_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
         MPI_Send(n_p, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
      }
   } else { /* my_rank != 0 */
      MPI_Recv(a_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
      MPI_Recv(b_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
      MPI_Recv(n_p, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
   }
}  /* Get_input */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

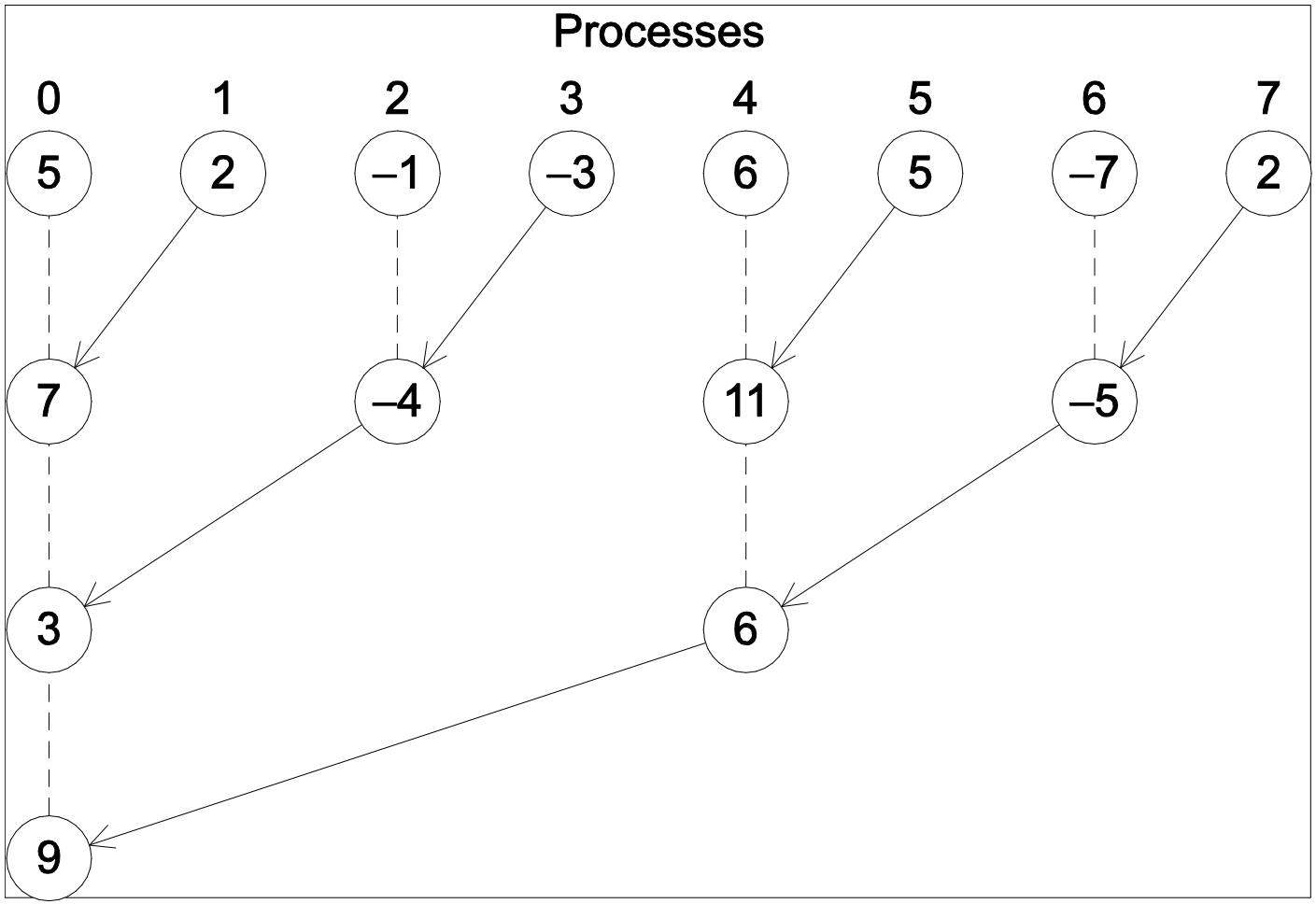
# Tasks and communications for Trapezoidal Rule




**Too much burden on the collector node**
**Can we better split the communication?**

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# A tree-structured global sum





Copyright © 2010, Elsevier Inc. All rights Reserved

# Tree-structured communication

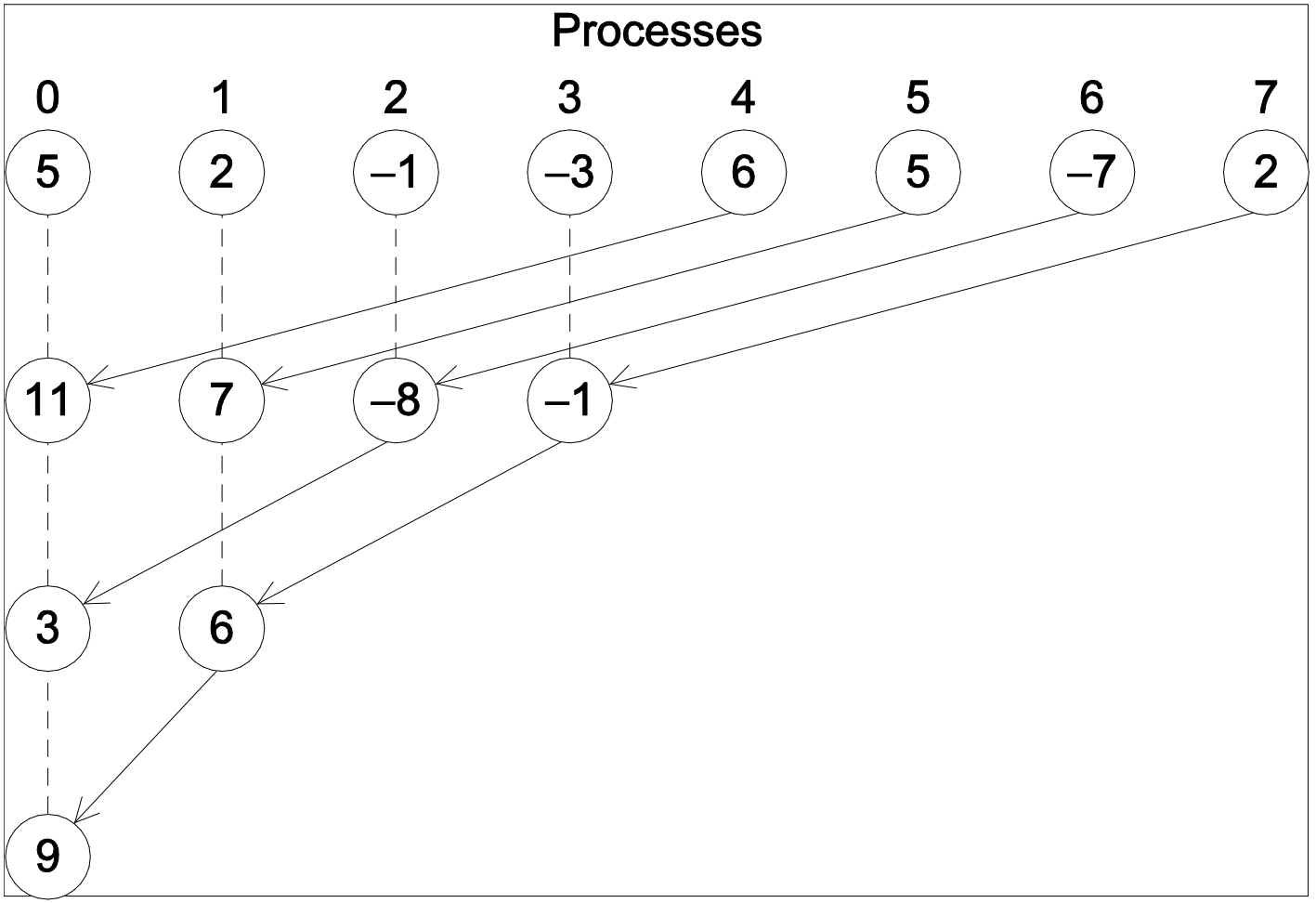
1. In the first phase:
   (a) Process 1 sends to 0, 3 sends to 2, 5 sends to 4, and 7 sends to 6.
   (b) Processes 0, 2, 4, and 6 add in the received values.
   (c) Processes 2 and 6 send their new values to processes 0 and 4, respectively.
   (d) Processes 0 and 4 add the received values into their new values.

2. (a) Process 4 sends its newest value to process 0.
   (b) Process 0 adds the received value to its newest value.

**Is that the only way?**

MK MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# An alternative tree-structured global sum




MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# MPI_Reduce

```
int MPI_Reduce(
      void*          input_data_p    /* in  */,
      void*          output_data_p   /* out */,
      int            count           /* in  */,
      MPI_Datatype   datatype        /* in  */,
      MPI_Op         operator        /* in  */,
      int            dest_process    /* in  */,
      MPI_Comm       comm            /* in  */);
```

```
MPI_Reduce(&local_int, &total_int, 1, MPI_DOUBLE, MPI_SUM, 0,
      MPI_COMM_WORLD);
```

```
double local_x[N], sum[N];
. . .
MPI_Reduce(local_x, sum, N, MPI_DOUBLE, MPI_SUM, 0,
      MPI_COMM_WORLD);
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

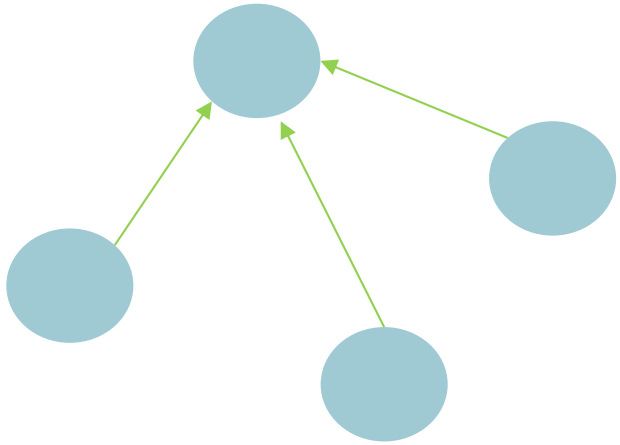
# Predefined reduction operators in MPI

| Operation Value | Meaning |
|---|---|
| MPI_MAX | Maximum |
| MPI_MIN | Minimum |
| MPI_SUM | Sum |
| MPI_PROD | Product |
| MPI_LAND | Logical and |
| MPI_BAND | Bitwise and |
| MPI_LOR | Logical or |
| MPI_BOR | Bitwise or |
| MPI_LXOR | Logical exclusive or |
| MPI_BXOR | Bitwise exclusive or |
| MPI_MAXLOC | Maximum and location of maximum |
| MPI_MINLOC | Minimum and location of minimum |

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Communication hands-on

Implement a “reduce” by using send and receive + average/max/min over the set of data collected




Copyright © 2010, Elsevier Inc. All rights Reserved

# Collective vs. Point-to-Point Communications

- All the processes in the communicator must call the same collective function.

- For example, a program that attempts to match a call to MPI_Reduce on one process with a call to MPI_Recv on another process is erroneous, and, in all likelihood, the program will hang or crash.

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Collective vs. Point-to-Point Communications

- The arguments passed by each process to an MPI collective communication must be "compatible."

- For example, if one process passes in 0 as the dest_process and another passes in 1, then the outcome of a call to MPI_Reduce is erroneous, and, once again, the program is likely to hang or crash.


Copyright © 2010, Elsevier Inc. All rights Reserved

# Collective vs. Point-to-Point Communications

- The output_data_p argument is only used on dest_process

- However, all of the processes still need to pass in an actual argument corresponding to output_data_p, even if it's just NULL.


Copyright © 2010, Elsevier Inc. All rights Reserved

# Collective vs. Point-to-Point Communications

- **Point-to-point** communications are matched on the basis of **tags** and **communicators**.
- **Collective** communications **don't use tags**.
- They're matched solely on the basis of the **communicator** and the **order** in which they're called.


Copyright © 2010, Elsevier Inc. All rights Reserved

# Example (1)

| Time | Process 0 | Process 1 | Process 2 |
|---|---|---|---|
| 0 | `a = 1; c = 2` | `a = 1; c = 2` | `a = 1; c = 2` |
| 1 | `MPI_Reduce(&a, &b, ...)` | `MPI_Reduce(&c, &d, ...)` | `MPI_Reduce(&a, &b, ...)` |
| 2 | `MPI_Reduce(&c, &d, ...)` | `MPI_Reduce(&a, &b, ...)` | `MPI_Reduce(&c, &d, ...)` |

Multiple calls to MPI_Reduce

Process 0 is acting as destination

What are the values in b and d for process 0?

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Example (2)

- Suppose that each process calls MPI_Reduce with operator MPI_SUM, and destination process 0.

- At first glance, it might seem that after the two calls to MPI_Reduce, the value of b will be 3, and the value of d will be 6.

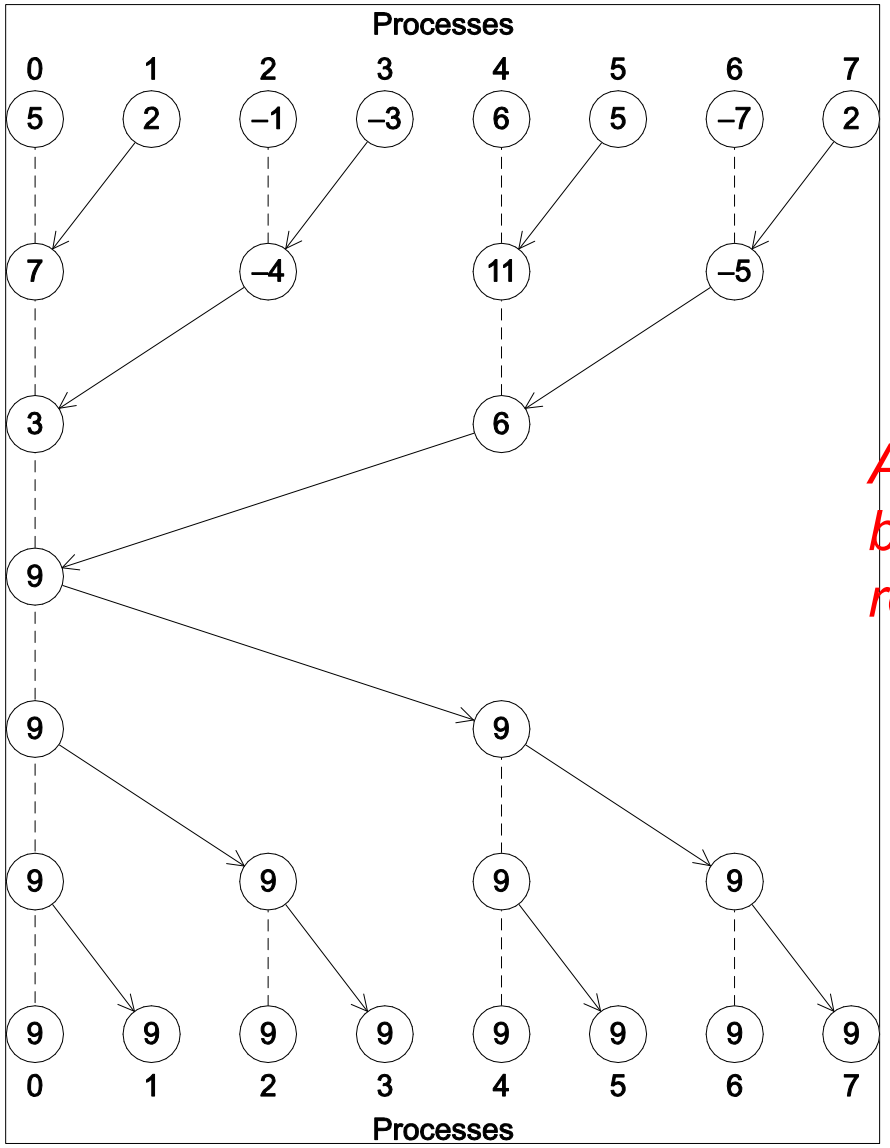
MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# MPI_Allreduce

- Useful in a situation in which all of the processes need the result of a global sum in order to complete some larger computation.

```
int MPI_Allreduce(
        void*        input_data_p   /* in  */,
        void*        output_data_p  /* out */,
        int          count          /* in  */,
        MPI_Datatype datatype       /* in  */,
        MPI_Op       operator       /* in  */,
        MPI_Comm     comm           /* in  */);
```

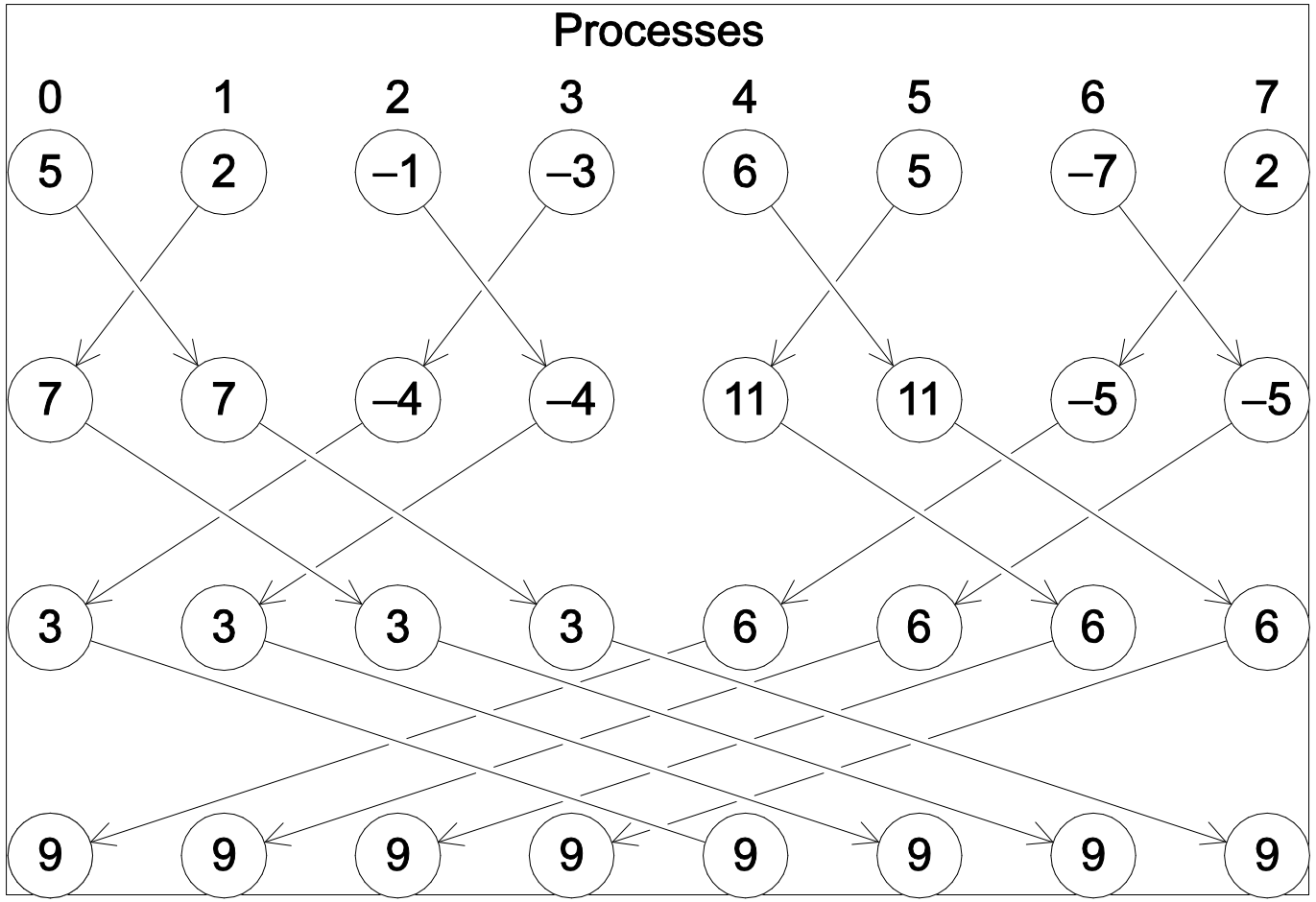
MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved




*A global sum followed by distribution of the result.*

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved






*A butterfly-structured global sum.*

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Broadcast

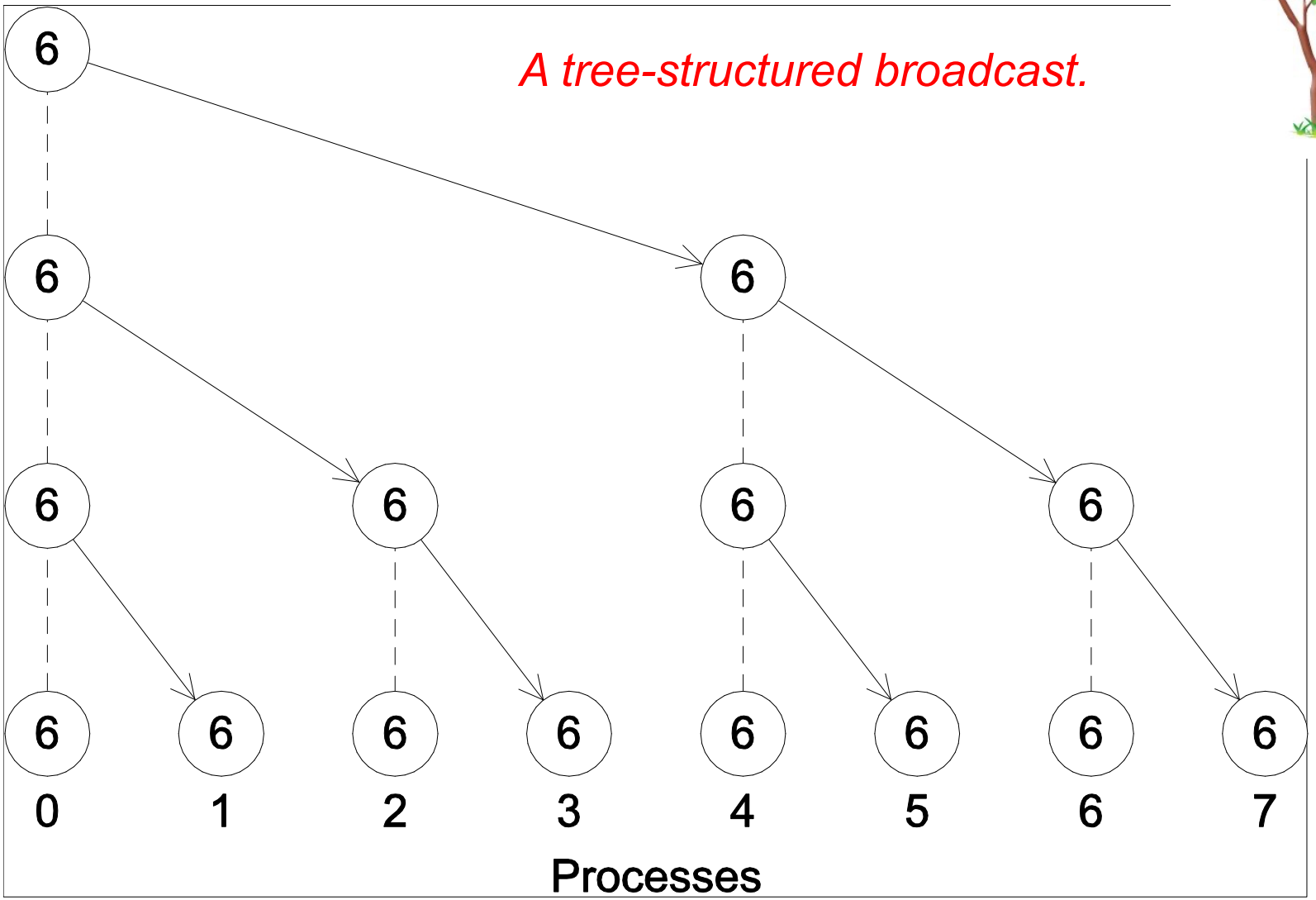
- Data belonging to a single process is sent to all of the processes in the communicator.

```
int MPI_Bcast(
      void*         data_p       /* in/out */,
      int           count        /* in     */,
      MPI_Datatype  datatype     /* in     */,
      int           source_proc  /* in     */,
      MPI_Comm      comm         /* in     */);
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

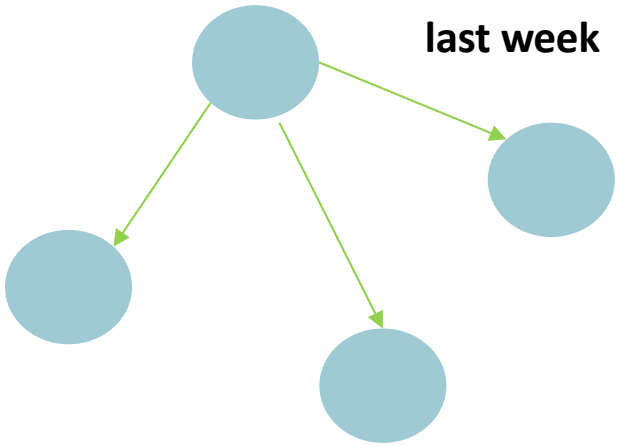





MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Get_input without MPI_Bcast

Exercise last week



```
void Get_input(
      int       my_rank   /* in  */,
      int       comm_sz   /* in  */,
      double*   a_p       /* out */,
      double*   b_p       /* out */,
      int*      n_p       /* out */) {
   int dest;

   if (my_rank == 0) {
      printf("Enter a, b, and n\n");
      scanf("%lf %lf %d", a_p, b_p, n_p);
      for (dest = 1; dest < comm_sz; dest++) {
         MPI_Send(a_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
         MPI_Send(b_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
         MPI_Send(n_p, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
      }
   } else { /* my_rank != 0 */
      MPI_Recv(a_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
      MPI_Recv(b_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
      MPI_Recv(n_p, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
            MPI_STATUS_IGNORE);
   }
}  /* Get_input */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Get_input that uses MPI_Bcast

```
void Get_input(
      int        my_rank   /* in  */,
      int        comm_sz   /* in  */,
      double*    a_p       /* out */,
      double*    b_p       /* out */,
      int*       n_p       /* out */) {

   if (my_rank == 0) {
      printf("Enter a, b, and n\n");
      scanf("%lf %lf %d", a_p, b_p, n_p);
   }
   MPI_Bcast(a_p, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
   MPI_Bcast(b_p, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
   MPI_Bcast(n_p, 1, MPI_INT, 0, MPI_COMM_WORLD);
}  /* Get_input */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Feedback form

- Throughout the entire HPC4DS course there will be a form available for your feedback
  - Please provide any comment about:
    - Pros
    - Cons
    - Aspects that were not clear enough during the class
    - Any other feedback you think can be relevant for the course
    - …
- Information are gathered in an anonymous way

https://docs.google.com/forms/d/e/1FAIpQLScSVpNPfMP8poNNz-nLl9xlKlHkeBM2x_JgWyA0qQu6mOBl5A/viewform

# Question time

# Feedback form

- Throughout the entire HPC4DS course there will be a form available for your feedback
  - Please provide any comment about:
    - Pros
    - Cons
    - Aspects that were not clear enough during the class
    - Any other feedback you think can be relevant for the course
    - …
- Information are gathered in an anonymous way

https://docs.google.com/forms/d/e/1FAIpQLScSVpNPfMP8poNNz-nLl9xlKlHkeBM2x_JgWyA0qQu6mOBl5A/viewform